

Question Bank — Week 11

Trees (Lecture 11) — Answers

DSA27 · Data Structures and Algorithms

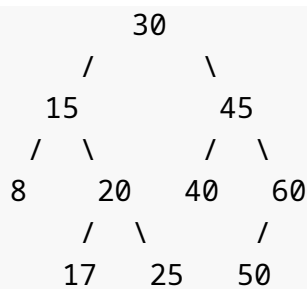
Fall 2026

Contents

Part A — Multiple choice	1
Part B — Short answer and essay	3
Part C — Trace the code	5
Part D — Tree state — draw every step	6
Part E — Complexity analysis	9
Part F — Find and fix the bug	10
Part G — Write the code	11

Questions: [week11-questions.md](#). Commit to your own answer before reading one here. Every walk, tree and count below was checked by running the reference `dsa/tree.py`.

The tree **Q**, built from `[30, 15, 45, 8, 20, 40, 60, 17, 25, 50]`:



Height 3; leaves 8, 17, 25, 40 and 50.

Part A — Multiple choice

Q	Ans	Q	Ans	Q	Ans	Q	Ans
M01	b	M07	c	M13	b	M19	b
M02	c	M08	d	M14	a	M20	a
M03	d	M09	d	M15	b	M21	c

Q	Ans	Q	Ans	Q	Ans	Q	Ans
M04	a	M10	b	M16	c	M22	d
M05	b	M11	a	M17	d		
M06	a	M12	c	M18	a		

W11-M01 — b. No children. (a) is the root; a node with one child (c) is internal.

W11-M02 — c. Height counts edges, and a single node has none. -1 (a) is the **empty** tree, which makes $1 + \max(-1, -1) = 0$ work for a leaf.

W11-M03 — d. Levels 0 to 3 hold at most $1 + 2 + 4 + 8 = 15 = 2^{h+1} - 1$. 7 (a) is height 2; 8 (b) is the size of level 3 alone.

W11-M04 — a. $\lfloor \log_2 100 \rfloor = 6$: height 6 holds up to 127 nodes, height 5 only 63. 99 (d) is the **tallest** possible height, a chain.

W11-M05 — b. The property is about whole subtrees. (a) is the weaker parent-child check, which W11-M20 shows is not enough; (c) is the heap property of Week 12.

W11-M06 — a. The root is 30, its left child 15; $20 > 15$ went right. 17 and 25 are 20's children; 8 is 15's left child.

W11-M07 — c. 30 (go left), 15 (go right), 20 (go right), 25 (found).

W11-M08 — d. The cost is $O(h)$, and nothing in `dsa/tree.py` keeps h small: sorted input makes $h = n - 1$. $O(\log n)$ (b) is true only while the tree stays short — on average for random input, or always for a balanced tree.

W11-M09 — d. 7 first, then 1, then each value between the last two: every node gets one child, a zigzag chain of height 6. (a) and (b) build the perfect tree of height 2 (W11-S2); (c) has height 3.

W11-M10 — b. Node, then the whole left subtree, then the right. (a) is in-order, (c) level-order, (d) post-order.

W11-M11 — a. Both subtrees before the node, so the root 30 is last. (c) is not any of the walks; (d) visits 8 after 17 and 25, which are in a different subtree.

W11-M12 — c. Left subtree (all smaller), node, right subtree (all larger): sorted, for every BST.

W11-M13 — b. Pre-order lists every node before its descendants, so re-inserting it sends each value down the same path to the same place. The in-order walk (a) is sorted and rebuilds a **chain**; post-order (c) inserts leaves first, so the first value inserted — a leaf — becomes the root.

W11-M14 — a. Post-order finishes a node's children before the node: free the children while you still hold the links to them, and apply an operator only after both operands have values.

W11-M15 — b. Recursion is last in, first out: it finishes the most recent node's subtree before returning. Level-order needs first in, first out — the nodes in the order they were

discovered. Memory (a) is not the reason; the queue can be larger than the stack (W11-K3).

W11-M16 — c. After dequeuing 45, and again after dequeuing 20, four nodes wait: 8, 20, 40, 60 and then 40, 60, 17, 25 (W11-S3).

W11-M17 — d. The in-order successor. The largest value of the right subtree (a) is too large: everything else in that subtree is smaller than it, yet would sit on its right.

W11-M18 — a. Right once to 45, then left to 40, which has no left child: 40 is the successor and goes to the root. 25 (b) is the **predecessor** — also a correct choice in general, but not `dsa/tree.py`'s.

W11-M19 — b. With no left child, the successor is a leaf or has one (right) child, so it is removed by the easy rule. It need not be a leaf (c): in Lecture 11's second example the successor 10 has a right child, 14.

W11-M20 — a. It accepts the lecture's tree $8 \rightarrow 3 \rightarrow 9$, where 9 is a right child of 3 but sits in 8's left subtree. It never rejects a valid BST, and it is $O(n)$ — but wrong.

W11-M21 — c. Left operand, right operand, operator: $2\ 3\ +\ 4\ *$, the postfix form of Lecture 06. (a) is pre-order; (b) is in-order, which has lost the brackets and means 14, not 20.

W11-M22 — d. Both keep extra data in each node — a height or a colour — and rotate nodes after changes so that the height stays $O(\log n)$. They are beyond the bylaw (named, not examined); `dsa/tree.py` is **unbalanced** (a).

Part B — Short answer and essay

W11-E1 (3)

- **Root:** the one node with no parent — 30. **Leaf:** a node with no children — 8, 17, 25, 40, 50. **Parent:** 20 is the parent of 17 and 25.
- **Subtree:** a node and everything below it — the subtree of 45 is {45, 40, 60, 50}.
- **Depth:** edges from the root down to the node — 17 has depth 3. **Height:** edges on the longest path down to a leaf — 15 has height 2 (15, 20, 17), and the tree's height is 3.
- $n - 1$ **edges:** every node except the root has exactly one parent, so exactly one edge comes into it from above; the root has none. Counting edges by the node they lead into gives $n - 1$.

W11-E2 (4)

- **Property:** for every node x , every value in x 's left subtree is smaller than x and every value in its right subtree is larger.
- **Counter-example:** root 8, left child 3, right child 10, and 9 as the right child of 3. Each pair is in order ($3 < 8$, $10 > 8$, $9 > 3$), but 9 is in 8's left subtree. `contains(9)` goes right at 8 and never finds it.

- **Design:** carry an allowed range (low, high) down the recursion, starting unbounded at the root. Going left from x sets $high = x$; going right sets $low = x$. A node outside its range makes the tree invalid; the empty tree is valid. Test “no bound” with `is not None`, not truthiness, or a bound of 0 is lost.
- **Cost:** each node is checked once, $O(1)$ work: $O(n)$ time, $O(h)$ stack.

W11-E3 (4)

- **Leaf:** set the parent’s link to it to `None`.
- **One child:** point the parent’s link at that child; the child’s whole subtree moves up one level and stays in order, because it lay on the same side of every ancestor as the deleted node.
- **Two children:** find the in-order successor — right once, then left until there is no left child. Copy its value into the node, then remove the successor’s node.
- **Why the successor fits:** it is larger than everything in the left subtree (they are smaller than the deleted value, which is smaller than it), and it is the smallest value of the right subtree, so smaller than all that remain there.
- **Why removal is easy:** it was reached by going left until there was no left child, so it has no left child: it is a leaf or has one right child — one of the easy cases. Watch the case where the successor is the node’s own right child: then the link to change is the node’s `right` (W11-B3).
- **Always:** keep the parent; if the node is the root, `self.root` changes. $O(h)$.

W11-E4 (4)

- **Pre-order** (node, left, right): serialising a tree — re-inserting the list rebuilds the same shape, because every node comes after its ancestors; also prefix notation.
- **In-order** (left, node, right): listing a BST’s values in **sorted** order without sorting; checking a BST (strictly increasing); range queries.
- **Post-order** (left, right, node): freeing a tree (children before the parent whose links you still need), evaluating an expression tree (operands before operator), computing sizes and heights (a node’s answer needs its children’s).
- **Level-order** (by depth, left to right): printing a tree level by level; the nearest nodes first — the idea of breadth-first search. Needs a queue.
- (Marks: one for the orders, one per two correct uses with reasons.)

W11-E5 (4)

- **True when the tree is short:** every operation walks one path, $O(h)$, and h can be as small as $\lfloor \log_2 n \rfloor$. On random insertion orders h is $O(\log n)$ on average — measured in Lecture 11, 26.6 for 4,096 values.
- **False when the tree is tall:** inserting sorted values (1, 2, ..., n) makes every new value the largest so far, so it always goes right: a chain of height $n - 1$, and search is $O(n)$ — about 205 μ s against 1.8 μ s at $n = 4,096$ in the lecture’s measurement.
- **Cure:** a balanced BST — AVL or red-black — which re-shapes itself with rotations to keep $h = O(\log n)$ in the worst case (beyond this course); or, with all values known in advance, insert the middle first (W11-C4).
- **Against a sorted array:** search $O(\log n)$ in both (the BST only while short); insert

$O(n)$ in the array (shifting) against $O(h)$ in the BST (no moves); listing in order $O(n)$ in both (in-order walk).

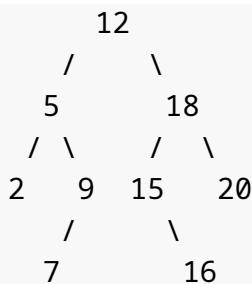
Part C — Trace the code

W11-T1

contains(25)	Decision	contains(42)	Decision
30	$25 < 30$: left	30	$42 > 30$: right
15	$25 > 15$: right	45	$42 < 45$: left
20	$25 > 20$: right	40	$42 > 40$: right
25	equal: True	None	False

contains(42) stepped off the right of 40, so insert(42) makes the new node **40's right child**. Both searches took 4 comparisons at most, $h + 1 = 4$.

W11-T2 The tree:



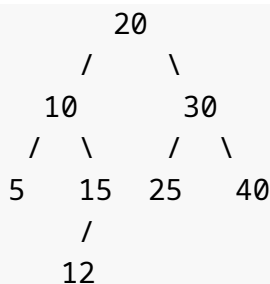
```

pre-order  12, 5, 2, 9, 7, 18, 15, 16, 20
in-order   2, 5, 7, 9, 12, 15, 16, 18, 20
post-order 2, 7, 9, 5, 16, 15, 20, 18, 12
level-order 12, 5, 18, 2, 9, 15, 20, 7, 16

```

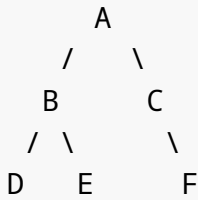
Two checks that need no drawing: the in-order walk is the values sorted; the level-order walk equals the insertion order here, because the values were listed level by level.

W11-T3 (a) Insert the pre-order list into an empty BST (that is exactly what pre-order is for):



Post-order: **5, 12, 15, 10, 25, 40, 30, 20.**

- (b) Pre-order says A is the root. In the in-order walk, what is left of A (D B E) is A's left subtree and what is right of it (C F) its right subtree. Repeat on each part: B is the left subtree's root (first in pre-order), with D on its left and E on its right; C is the right subtree's root, with nothing on its left and F on its right.



Post-order: **D, E, B, F, C, A.**

Why two walks for (b): in a BST the order of the values tells you, at every node, which side each value belongs to — the in-order walk is always just the sorted values, so pre-order alone is enough. For a general binary tree nothing says whether B is left or right of A; the in-order walk supplies exactly that. Pre-order alone would fit many trees: A with the chain B, D, E, C, F hanging to the left, for instance.

W11-T4 $f(\text{root}) = 185$: the largest sum of values along a root-to-leaf path, $30 + 45 + 60 + 50$. $g(\text{root}, k)$ for $k = 0 \dots 4 = \mathbf{1, 2, 4, 3, 0}$: the number of nodes at depth k — $\{30\}$; $\{15, 45\}$; $\{8, 20, 40, 60\}$; $\{17, 25, 50\}$; and none at depth 4. $g(\text{root}, k)$ is 0 exactly when k is larger than the height.

W11-T5

```

_is_valid(20, None, None)
  _is_valid(10, None, 20)
    _is_valid(5, None, 10)
      _is_valid(None, None, 5) -> True
      _is_valid(None, 5, 10)   -> True
    -> True
    _is_valid(25, 10, 20)      -> False   (25 >= 20)
  -> False
-> False
  
```

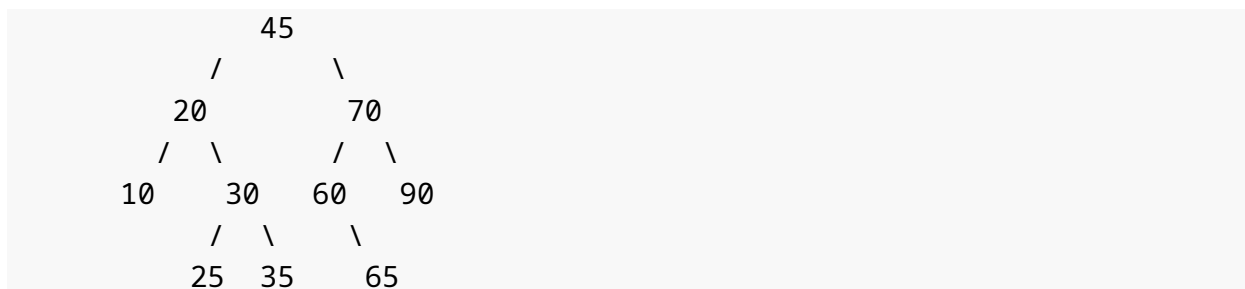
The call on 25 returns False first: 25 is a right child of 10, which is fine for its parent, but it is in 20's left subtree, so its range is (10, 20). The and stops there, so 30 is never visited, and `is_valid()` returns False.

Part D — Tree state — draw every step

W11-S1 The inserts (height after each):

Insert	Where it goes	Height
45	root	0
20	45.left	1

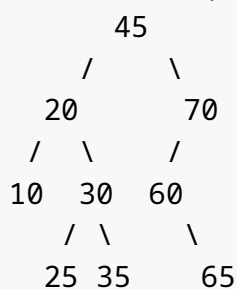
Insert	Where it goes	Height
70	45.right	1
10	20.left	2
30	20.right	2
60	70.left	2
90	70.right	2
25	30.left	3
35	30.right	3
65	60.right	3



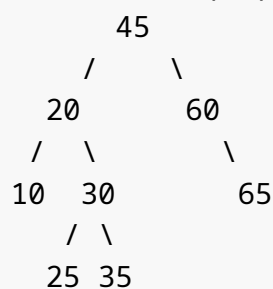
The deletes:

- **delete(90)** — a **leaf**: 70.right = None.
- **delete(70)** — now **one child**, 60: 45.right = 60, and 60 keeps its right child 65.
- **delete(45)** — the root, **two children**. Successor: right once to 60; 60 has no left child, so 60 **itself** is the successor. Copy 60 into the root; the old 60 has one child, 65, which takes its place: the root's right becomes 65.
- **delete(20)** — **two children**. Successor: right once to 30, then left to 25, a leaf. Copy 25 into 20's node; 30.left = None.

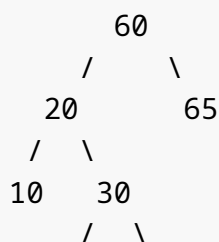
after delete(90)



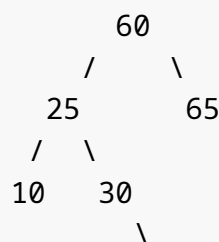
after delete(70)



after delete(45)



after delete(20)



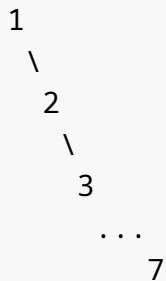
25 35

35

Every step leaves a valid BST (checked with `is_valid()`); the final pre-order is 60, 25, 10, 30, 35, 65, and the final height is 3.

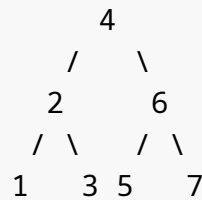
W11-S2

(a) 1, 2, 3, 4, 5, 6, 7



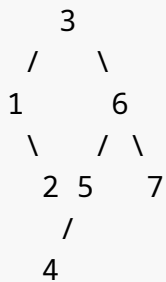
height 6

(b) 4, 2, 6, 1, 3, 5, 7



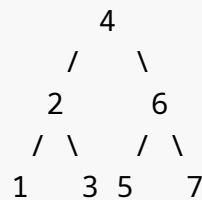
height 2

(c) 3, 1, 2, 6, 5, 7, 4



height 3

(d) 4, 6, 5, 7, 2, 3, 1



height 2

(b) and (d) build the **same tree**, and both have pre-order **4, 2, 1, 3, 6, 5, 7**. They must: the shape of a BST depends only on which values arrive before which **on each path**. In both orders 4 comes first, 2 comes before 1 and 3, and 6 before 5 and 7; the relative order of the two subtrees' values does not matter. (Pre-order is also how you check: equal pre-order walks mean equal BSTs.)

W11-S3

Dequeued	Queue after the step
(start)	30
30	15, 45
15	45, 8, 20
45	8, 20, 40, 60
8	20, 40, 60
20	40, 60, 17, 25
40	60, 17, 25
60	17, 25, 50
17	25, 50
25	50
50	(empty)

Level-order: 30, 15, 45, 8, 20, 40, 60, 17, 25, 50.

With a stack, pushing the right child before the left, the left child is always on top, so the walk finishes a node's whole left side before its right: **30, 15, 8, 20, 17, 25, 45, 40, 60, 50** — **pre-order**. Same loop, different container, different order: the difference between BFS and DFS (Week 14).

Part E — Complexity analysis

W11-K1

- (a) $\Theta(n^2)$. The i -th value is the largest so far and walks the whole chain of $i - 1$ nodes: $0 + 1 + \dots + (n - 1) = n(n - 1)/2$ comparisons.
- (b) $\Theta(n \log n)$ **on average**. In a randomly built BST the average node depth is about $2 \ln n \approx 1.39 \log_2 n$, so each insert walks $O(\log n)$ nodes.
- (c) **Tree sort**: the in-order walk is $\Theta(n)$, so the cost is the building: $\Theta(n \log n)$ in the best case (a short tree, e.g. middle first), $\Theta(n^2)$ in the worst (sorted or reverse-sorted input — the input that is already sorted is the one it handles worst).

W11-K2 The fault is calling height on the taller child **twice**: once in the comparison and again in the return.

- **Right chain of n nodes**. Each call on a node makes one call on the empty left side and **two** on the chain of $n - 1$ below: $C(n) = 2C(n - 1) + 2$, $C(0) = 1$, which solves to $C(n) = 3 \cdot 2^n - 2$: $\Theta(2^n)$. For $n = 20$: **3,145,726 calls** — counted by running it — where a correct height makes 41.
- **Perfect tree of height h** . Three calls on subtrees of height $h - 1$ (both sides are equal, so the return recomputes the right one): $C(h) = 3C(h - 1) + 1$, which solves to $C(h) = (3^{h+2} - 1)/2$: $\Theta(3^h) = \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$. For $n = 1,023$ ($h = 9$): 88,573 calls.
- **Fix**: compute each child's height once, into variables, and return $1 + \max(\text{left}, \text{right})$ — two calls per node, $\Theta(n)$. The same trap as fib in Lecture 03: a recursion that repeats work multiplies it.

W11-K3

	recursive in-order: $O(h)$ stack	level_order: $O(w)$ queue
perfect tree, n nodes	$\lfloor \log_2 n \rfloor + 1$ frames — tiny	up to $(n + 1)/2$ nodes (the last level)
chain of n nodes	n frames — and RecursionError near 1,000	1 node

On a short, bushy tree the recursive walk is far cheaper in memory; on a chain the queue is. Both are $\Theta(n)$ in time either way. The recursion's memory is the call stack, which Python

limits to about 1,000 frames; the queue's is ordinary memory in your CircularQueue, limited only by the machine.

Part F — Find and fix the bug

W11-B1. The loop walks until node is None, and then `node = TreeNode(value)` only rebinds a local variable: no link in the tree changes. Even the first insert is lost — `self.root` is never set, so `BinarySearchTree([8, 3, 10])` is empty, with root still None. **Fix:** handle the empty tree (`self.root = TreeNode(value)`), and stop **at the parent:** before stepping to a child, if it is None, set `node.left` or `node.right` to the new node and return.

W11-B2. It checks each node only against its own children. The tree 8, with 3 and 10 as children and 9 as the right child of 3, passes every check, but 9 is in 8's left subtree: not a BST. **Fix:** carry the range (`low`, `high`) allowed by all the ancestors down the recursion, as in `dsa/tree.py`'s `_is_valid` (W11-E2, W11-T5).

W11-B3. `succ_parent.left = succ.right` assumes the successor is the **left** child of its parent. That holds when the “go left” loop runs at least once, but when the successor is the node's own **right child** the loop runs zero times, `succ_parent` is the node itself, and the link to change is its right. On Q, `delete(20)`: the successor is 25, 20's right child. The code copies 25 into 20's node and then sets that node's **left** to None — so 17 vanishes, and the old 25 is still there as the right child:

```
in-order before: 8, 15, 17, 20, 25, 30, 40, 45, 50, 60
in-order after:  8, 15, 25, 25, 30, 40, 45, 50, 60
is_valid():      False
```

The tests' two-children case (deleting 3 from the sample tree) has a deep successor, so it passes; deleting the root 8 (successor 10, the right child) is what fails. **Fix:**

```
if succ_parent is node:
    succ_parent.right = succ.right
else:
    succ_parent.left = succ.right
```

— or, as `dsa/tree.py` does, move node and parent down to the successor and let the one-child code, which asks `parent.left is node`, do the removal.

W11-B4. `out = []` inside the helper rebinds `out` to a new, local list at every call, so every `append` goes into a list that is thrown away. The caller's list is never touched: `in_order()` always returns `[]`, and `test_in_order_is_sorted` fails with `assert [] == [1, 3, 4, 6, ...]`. **Fix:** delete that line; the helper must append to the list it was given.

Part G — Write the code

W11-C1

```
def count_leaves(node):
    if node is None:
        return 0
    if node.left is None and node.right is None:
        return 1
    return count_leaves(node.left) + count_leaves(node.right)
```

A leaf answers 1, the empty tree 0, and every other node the sum of its two subtrees' answers. One visit per node: $O(n)$. On Q: 5.

W11-C2

```
def is_balanced(node):
    return _balanced_height(node) != -2

def _balanced_height(node):
    # The height of the subtree, or -2 if it is unbalanced anywhere.
    if node is None:
        return -1
    left = _balanced_height(node.left)
    if left == -2:
        return -2
    right = _balanced_height(node.right)
    if right == -2 or abs(left - right) > 1:
        return -2
    return 1 + max(left, right)
```

The helper returns the height when the subtree is balanced and -2 — an impossible height — as soon as it finds a node that is not, and every caller passes the -2 straight up without further work. Each node is visited once: $O(n)$. A separate height at every node would recompute each subtree's height once for every ancestor: $O(n \log n)$ on a balanced tree, $O(n^2)$ on a chain — and the test, which counts reads on 8,191 nodes, measures 212,992 reads for that approach against a budget of 32,768.

W11-C3

```
def range_values(node, low, high):
    out = []
    _collect(node, low, high, out)
    return out

def _collect(node, low, high, out):
```

```

if node is None:
    return
value = node.value
if value > low:                # answers may lie on the left
    _collect(node.left, low, high, out)
if low <= value <= high:
    out.append(value)
if value < high:              # answers may lie on the right
    _collect(node.right, low, high, out)

```

It is the in-order walk with two ifs. If $\text{value} \leq \text{low}$, everything in the left subtree is smaller than value, hence below low: skip it. If $\text{value} \geq \text{high}$, the right subtree is above high: skip it. What remains is visited in in-order, so the answers come out sorted. Cost $O(h + k)$ for k answers: the walk follows at most two root-to-leaf boundary paths plus the subtrees that are entirely inside the range.

W11-C4

```

def build_balanced(values):
    return _build(values, 0, len(values) - 1)

def _build(values, lo, hi):
    if lo > hi:
        return None
    mid = (lo + hi) // 2
    root = TreeNode(values[mid])
    root.left = _build(values, lo, mid - 1)
    root.right = _build(values, mid + 1, hi)
    return root

```

The middle value is the root; the two halves, built the same way, are its subtrees. Each half has at most half the values, so the height is $\lfloor \log_2 n \rfloor$ — the smallest possible. Passing lo and hi avoids slicing, which would copy $O(n)$ values per level. $O(n)$ in total: one `TreeNode` per value. With all values known and sorted in advance, this is the “middle first” cure of Lecture 11.

W11-C5

```

def lowest_common_ancestor(node, a, b):
    while node is not None:
        value = node.value
        if a < value and b < value:
            node = node.left
        elif a > value and b > value:
            node = node.right
        else:
            return value
    # a and b split here

```

```
raise ValueError("a and b must be in the tree")
```

While a and b are both smaller than the node, both are in its left subtree, so the answer is there too; both larger, the right subtree. The first node where they are **not** on the same side — one is smaller and one larger, or one of them **is** the node — has a in one subtree (or is a) and b in the other (or is b), so no lower node can hold both. One path down: $O(h)$, with no recursion.